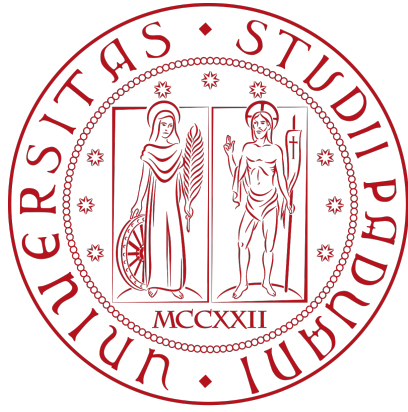




UNIVERSITÀ DEGLI STUDI DI PADOVA
Department of Mathematics “Tullio Levi-Civita”
Master’s Degree in Computer Science
Academic year 2025/2026

CPS and IoT Security

A lightweight distributed anomaly-based
Intrusion Detection System for
RPL networks in Contiki-NG and Cooja

Reference paper: B. Farzaneh, M. A. Montazeri, S. Jamali,
*An Anomaly-Based IDS for Detecting Attacks in RPL-Based
Internet of Things*, ICWR 2019

Author: **Riccardo Fabbian**

Student ID: **2140711**

Period of work: **September 2026**

1 Introduction and Objective

RPL (RFC 6550) is the routing protocol for Low-Power and Lossy Networks. Its security problem is that encryption protects a network from outsiders but not from a node that has already joined: such a node holds valid keys, so every message it sends is legitimate and cryptography cannot distinguish it from an honest one. The only remaining defence is to observe behaviour, which is what the reference paper does.

The paper detects two internal attacks specific to RPL, the **neighbour attack** and the **DIS attack**, using a threshold on the number of control messages each node hears from each of its neighbours. Its appeal is that it is genuinely lightweight and fully distributed: no central monitor, no reporting traffic. The authors report a True Positive Rate (TPR) of 100% with 0% False Positive Rate (FPR) for the DIS attack, and 94–100% TPR with 0.2–9.6% FPR for the neighbour attack.

This project has two objectives. The first is to **reproduce** the detector and both attacks on Contiki-NG v5.2 with Cooja, and to run a subset of the paper’s experimental matrix. The second is an **original extension**: the paper evaluates its counters once every five minutes, so an attack starting just after an evaluation goes unnoticed for almost a whole window. A second detector mode with a shorter sliding window was implemented to measure what faster detection costs.

2 Background and Threat Model

RPL organises the network as a tree (a DODAG) rooted at a border router. Each node holds a **rank**, its distance from the root, and forwards data to a **preferred parent** of lower rank. Two of the three control messages matter here. A **DIO** announces a node’s identity and rank, and is how the tree is built and maintained. A **DIS** solicits a DIO, and is normally sent only by a node that has just been switched on.

DIO transmission is paced by the **Trickle** algorithm: frequent while the network is unstable, then doubling the interval until the network is almost silent. Crucially, *receiving a multicast DIS resets the Trickle timer to its minimum*. These two facts are the basis of everything that follows. Because a converged network is quiet, a node sending many DIOs is a statistical outlier, which is what the detector exploits. Because a DIS resets everyone’s timer, a DIS attacker makes honest nodes do the work for it, which is why Section 5 shows it to be by far the more expensive attack.

The attacker is an **internal** node that joined legitimately and sends well-formed messages; it does not break link-layer security. In the **neighbour attack** it floods DIOs, forcing neighbours to reprocess and reconsider parent selection. In the **DIS attack** it floods solicitations, provoking a storm of DIOs from honest nodes. Every node runs the detector on what its own radio hears, with no cooperation. As in the paper, wormhole, sinkhole, selective-forwarding and rank attacks are out of scope.

3 The Reference Model

The paper’s detector repeats four phases per observation window. **(I)** Each node counts the DIOs received from every neighbour. **(II)** At the end of the window it computes the mean μ and standard deviation σ of those counts and derives a threshold $T = \mu + k\sigma$, where the coefficient

$$k = -5 \cdot 10^{-5}x^4 + 0.0037x^3 - 0.0899x^2 + 0.9281x - 0.7903 \quad (1)$$

depends on the number of neighbours x . The authors obtained it by simulating attack-free networks, recording for each neighbourhood size the largest deviation occurring naturally, and fitting a curve through those maxima. **(III)** A neighbour above T is a neighbour attacker; independently, a neighbour whose DIS count exceeds 3 is a DIS attacker, 3 being the maximum the authors ever observed in normal traffic. **(IV)** A detected neighbour is **temporarily blocked** for one minute, becoming permanently blocked once a counter of repeated detections exceeds 2. The block is temporary precisely because an honest node may be flagged by accident.

3.1 What was reproduced, and where this setup differs

Reproduced from the paper: the profile, the dynamic threshold and its polynomial, the DIS threshold of 3, the four phases, temporary and permanent blocking, 20/30/40-node networks with 1, 20%

and 30% attackers, a 20 m grid, UDGM with 50 m transmission and 100 m interference range, constant-bit-rate UDP toward the root, 30-minute runs, attacks starting at 75 s, a 5-minute window, and `block_threshold` = 2 with a 1-minute temporary block. Each configuration is repeated over 5 seeds, as in the paper.

The differences are stated here rather than left implicit, because several of them bear on the results.

- **Deployment area.** The paper specifies 100×100 m. The generator places the root at top centre and the remaining nodes on a five-column grid at 20 m spacing, which gives 80×80 m at 20 nodes, 80×120 m at 30 and 80×160 m at 40 (Figure 1). The 40-node network is therefore a 2:1 strip rather than a square, and is deeper in hops than the paper’s.
- **Inter-node distance.** The paper sweeps 20, 30 and 40 m; only 20 m was run here. The matrix is consequently one column of the paper’s, not all of it.
- **Attacker proportions.** One attacker at 20 nodes, 20% at 30 nodes, 30% at 40 nodes — three of the paper’s nine (size, proportion) pairs, not the full cross.
- **Attack rate and duration.** Attacks last 600 s (75–675 s) rather than to the end of the run. The main matrix uses a fixed injection period of 10 s at 20 and 30 nodes and 5 s at 40 nodes; the paper’s DIS attacker instead sends randomly every 5–60 s, which is used in the rate sweep and the DIS-threshold study of Section 5.4. The main-matrix DIS attacker is thus about three times faster than the paper’s.
- **MAC layer.** The paper uses ContikiMAC duty cycling, which no longer exists in Contiki-NG v5.2; CSMA with the radio always on is used instead. Energy is therefore not measured, and the cost of the attacks is reported as control traffic rather than as duty cycle.
- **Observed window count.** In paper mode a 30-minute run completes five of its six 300 s evaluations: the sixth would fall at 1801 s, just past the 1800 s timeout. Detection statistics are unaffected, since the attack ends at 675 s.

4 Implementation

The system is four Contiki-NG firmwares plus a host-side analysis pipeline, all pinned inside a Docker image (Contiki-NG `release/v5.2`, commit `4ccd1a3b`) so that results are reproducible. Cooja runs headless, driven by a control script inside each scenario, which is what makes a matrix of 168 simulations feasible. Simulations use the native `cooja` mote: 30 simulated minutes of a 40-node network cost about 22 s of wall clock end to end, of which Cooja reports 8.4 s of actual simulation, a speed-up of about $210\times$. The Tmote Sky target is used only for the memory measurements of Section 5.5.

4.1 Capturing and Filtering Control Messages

Contiki-NG registers exactly one handler per ICMPv6 message type, so a second handler cannot be added from application code. A small patch to the routing layer, applied automatically at image build time, inserts a hook at two points: the top of the DIS handler, and the DIO handler immediately after the sender address becomes available. Both call the same hook and differ only in the message code and in how the sender address is reached:

```
/* dis_input(), before RPL parses the message */
if(!ids_rpl_input(RPL_CODE_DIS, &UIP_IP_BUF->srcipaddr)) goto discard;

/* dio_input(), once the sender address is available */
if(!ids_rpl_input(RPL_CODE_DIO, &from)) goto discard;
```

This single hook serves both purposes of the design: it is where messages are counted and where blocked neighbours are filtered. Filtering here rather than deleting the neighbour from the routing tables is deliberate: it achieves the intended effect, since a discarded DIS cannot reset a Trickle timer and a discarded DIO cannot make the attacker a parent, without risking corruption of routing state that the routing layer would in any case rebuild from the next message.

4.2 The Detector

Each node keeps its own neighbour table, independent of the routing layer’s. This is an interpretation: the paper does not say which “neighbour list” it means, and the routing layer only tracks candidate

parents whereas the detector must count every node it hears. Both counts are logged so the difference can be inspected. The hook itself is short:

```
int ids_rpl_input(uint8_t code, const uip_ipaddr_t *from) {
    n = lookup(ids_node_id(from), 1);
    if(code == RPL_CODE_DIO) n->dio++; else if(code == RPL_CODE_DIS) n->dis++;
    if(block_state(n) != 0) { n->dropped++; return 0; } /* filtered */
    return 1;
}
```

The ordering is essential and easy to get wrong: **counting happens before the block is checked**. Were a blocked neighbour to stop being counted, it would appear silent, would never be detected again, and would never reach the permanent block, so the paper’s escalation would never fire.

The threshold is evaluated in **integer arithmetic scaled by 1000**, because the target has no floating-point unit. The polynomial for k is never evaluated on the mote: since the neighbour count is a small integer, all 41 values are precomputed on the host and compiled in as a lookup table costing 82 bytes. The standard deviation uses an exact integer square root, and the variance is obtained from the identity $n^2\sigma^2 = n \sum x_i^2 - (\sum x_i)^2$, which avoids any intermediate rounding:

```
num = (uint64_t)n * sumsq - (uint64_t)sum * sum; /* n^2 * variance */
mean_x1000 = (sum * 1000ULL) / n;
sigma_x1000 = isqrt64(num * 1000000ULL) / n;
thr_x1000 = mean_x1000 + (k_x1000 * sigma_x1000) / 1000;
```

The sliding mode reuses this function unchanged. It differs in two respects: each neighbour keeps six 10-second buckets in a ring buffer instead of one counter, so the *observation window* is 60s rather than 300s, and the evaluation timer fires every 10s rather than every 300s, so the *cadence* is thirty times higher. Both changes matter, and Section 5.4 shows that the shorter window, not the faster cadence, is what breaks the DIS rule. Confining the difference to those two points is what makes the comparison in Section 5.3 meaningful, since both modes run identical detection code on identical topologies and seeds.

4.3 Attackers, Logging and Verification

The attacker firmwares are ordinary nodes plus a timer that calls the routing layer’s *own* DIO or DIS transmission functions. Using the protocol’s functions rather than crafting packets guarantees the injected messages are indistinguishable from legitimate ones, which is what the threat model requires. Rate, start and duration are build parameters.

Nodes emit tab-separated records over the serial line: one per received message, one per neighbour at each evaluation, one for the computed profile and threshold, one per alert and per blocking action, plus periodic status lines. Attackers additionally log every injection, and the scenario generator writes a separate ground-truth file naming the attackers. **The analysis joins alerts against that file and never infers ground truth from detector output**, which would make the evaluation circular.

Because the mote arithmetic is hand-written integer code, it is duplicated in Python as a bit-exact reference model. Two checks use it. A unit suite of 28 tests covers the awkward cases: zero neighbours, one neighbour, identical counts, a single large outlier, and DIS counts exactly at and one above the threshold. Separately, `scripts/replay_check.py` replays *every* stored run log, recomputing the profile and the alert set from the logged per-neighbour counts and requiring an exact match on every intermediate value; over the whole `logs/` directory this checks 306 841 monitor–window decisions with no disagreement. Cooja is deterministic for a given seed and firmware: the whole matrix was regenerated from scratch, rebuilding the firmware and re-simulating all 168 scenarios, and every log came back byte-identical to the one it replaced. All variation reported below therefore comes from changing the seed, not from run-to-run noise.

5 Results and Critical Evaluation

The main matrix pairs each network size with one attacker proportion — 20 nodes with a single attacker, 30 nodes with 20%, 40 nodes with 30% — and runs both attacks and both detector modes over 5 seeds, together with an attack-free baseline at each size. Two sweeps, on the DIS threshold

and on the attack rate, use 3 seeds. Figure 1 shows the largest configuration, generated from the seed together with its ground-truth file. Node-level metrics classify each node once, matching the paper’s table: an attacker is a true positive if any monitor flagged it, a normal node a false positive if any monitor flagged it *with the same rule*. That last qualification follows the paper’s per-attack FPR columns, and Section 5.1 shows it is not innocent.

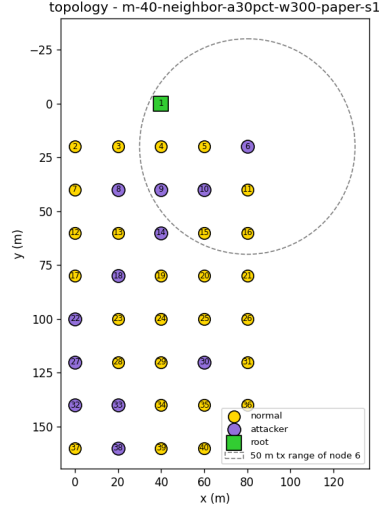


Figure 1: The 40-node scenario with 30% attackers, on the five-column 20 m grid actually used (80×160 m). Root at top centre (green), attackers in purple; the dashed circle is one attacker’s 50 m transmission range

5.1 Baseline and the DIS Attack

Attack-free runs converge within about 60 s, deliver every datagram, and average one parent change per node (1.00 at 20 nodes, 1.01 at 30 and 40): every node picks a parent once and keeps it. Per-neighbour DIO counts fall from the Trickle burst — a mean of 6.8 and a maximum of 11 in the first window — to a mean of 0.8 and a maximum of 5 thereafter. The per-neighbour DIS count never exceeded 2 in any window of any baseline run, and was 0 after the first window, which independently supports the paper’s choice of 3 as a safe threshold.

The DIS attack reproduces the paper’s headline result: 100% TPR and 0% FPR at all three size/proportion pairs, every seed and both modes. Figure 2 shows why the result is so clean rather than merely good: honest nodes stop sending DIS after joining, the attacker never stops, and the two populations do not overlap at all, so a threshold placed between them cannot err in either direction. This is the ideal case for a threshold detector, and the paper’s headline rests on that separation rather than on the sophistication of the method.

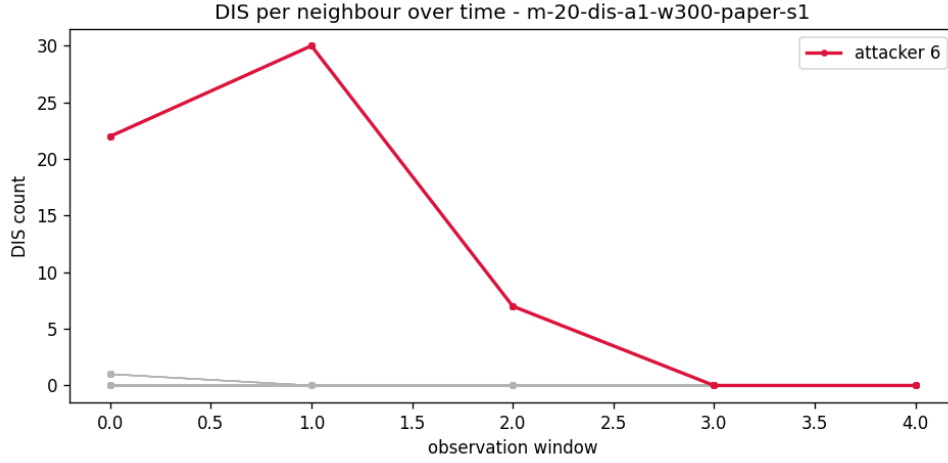


Figure 2: DIS messages per neighbour per 300s window, 20 nodes with one DIS attacker; the attack runs from 75 to 675s, so the attacker falls silent after window 2. Honest nodes stop soliciting once they have joined, and while the attack is on the two populations are separated by a wide margin

That 0% deserves one caveat, because it is a property of the DIS *rule* and not of the detector as deployed. In the same runs the independent DIO rule does fire on honest neighbours. Averaged over the five seeds of the 20-node DIS scenario, paper mode issues 4.2 temporary and 0.2 permanent blocks against honest neighbours; the sliding mode issues 197 temporary and 74 permanent, and over a run it blocks every one of the 18 honest neighbours it hears at least once. Scored per rule, as the paper scores it, the DIS column is a clean 0; scored by what the node actually does to its neighbours, it is not.

5.2 The Neighbour Attack and Two Structural Findings

A single neighbour attacker is detected at 100% TPR. Figure 3 shows the mechanism at a detecting monitor: the attacker’s count crosses the dynamic threshold while the neighbourhood mean stays low.

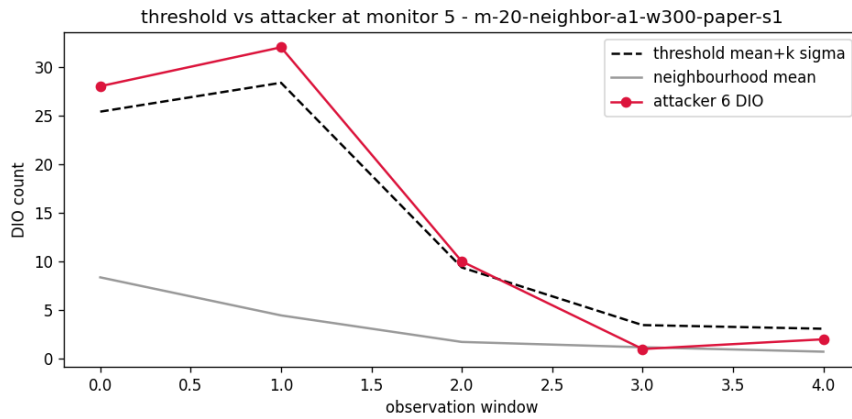


Figure 3: Dynamic threshold, neighbourhood mean and the attacker’s DIO count at a detecting monitor

But TPR **collapses as the attacker proportion grows**: 100% with one attacker, 33% at 20%, and 5% at 30%, against 94–99.7% in the corresponding cells of the paper. This is not an implementation defect, and the analysis of why is the main critical contribution of this work.

Finding 1 — blind spots at certain neighbourhood sizes. Algorithm 1 divides by the number of neighbours, making it the *population* standard deviation, computed over a set that *includes the attacker*. By Samuelson’s inequality no member of a sample of n values can lie further than $\sqrt{n-1}$ standard deviations from the mean, so a lone attacker is detectable only where $\sqrt{n-1} > k(n)$. This is a hard limit independent of attack rate, because every extra message the attacker sends raises the

mean and the deviation that define its own threshold. Evaluating the two curves over the whole table gives two blind bands, and they have different standing:

- $n = 5, 6, 7$ lies inside the range the paper actually fitted. Here k was fitted to naturally occurring maxima, which obey the same inequality, so the fitted curve necessarily runs close to the bound and crosses it. At $n = 7$ the margin is $k = 2.4504$ against $\sqrt{6} = 2.4495$, four parts in ten thousand. **This band was confirmed experimentally.**
- $n = 28 \dots 32$ is an artefact of the polynomial, not of the data. The paper’s Figure 5 plots its measured coefficients only for $x \leq 20$, where they rise to about 2.7 and flatten; Equation 1 does not pass through those points ($k(20) = 3.41$ against a measured 2.71) and, extrapolated beyond them, climbs to $k(32) = 5.66$ while $\sqrt{31} = 5.57$. The band exists only because a quartic is being evaluated far outside its fitted domain. **It could not be tested here:** on this grid a monitor hears at most 20 neighbours, and no monitor in any of the 168 runs exceeded that.

The lower band was confirmed directly. In a 10-node development scenario (30 m grid, 60 s window, one attacker injecting every 5 s) the attacker was heard by exactly three monitors. The monitor with 8 neighbours saw 12–13 attacker DIOs per window against a neighbourhood mean near 1, and flagged it in eight of the nine windows; the monitors with 5 and with 6 neighbours saw the same traffic and never flagged it once. The distributed design is what rescues the system, since a single non-blind monitor suffices, which is exactly why node-level TPR stays at 100% while individual monitors fail.

Finding 2 — attackers mask each other. When several attackers share a monitor they all inflate the mean and the deviation, so the threshold rises with them and none stands out. This generalises the same inequality to several outliers and explains the collapse at 20% and 30%. The measured drop is steeper than the paper’s, and three differences plausibly account for the gap: attackers here are placed by seed and often fall within range of one another; this attacker floods at a fixed rate whereas the paper’s rebroadcasts received DIOs, a less synchronised pattern; and the 40-node topology is an elongated strip rather than a square (Section 3.1), which concentrates each monitor’s neighbourhood. None of these changes the mechanism, so the conclusion stands as a limit of the published threshold under collusion.

False positives on the neighbour rule are not noise. They are quantisation: the counts are small integers, so a neighbour heard two or three times in a window where the others were heard once or not at all sits several deviations above a mean below one. Across every paper-mode neighbour and baseline run, the count that triggered a false positive was 1 in 28% of cases, 2 in 35%, 3 in 17% and 4 or more in the remaining 20% — more than a quarter of all false alarms are a neighbour that was heard exactly once, against a neighbourhood mean near 0.1. This effect is the whole source of the neighbour-rule FPR, here a few per cent, and very probably of the paper’s 0.2–9.6% as well.

5.3 Fixed versus Sliding Window

Nodes	Attack	Att.	Mode	TPR %	FPR %	Lat. s	DIO/min
20	—	0	paper	—	41.0	—	4.1
30	—	0	paper	—	43.3	—	4.6
40	—	0	paper	—	51.0	—	4.9
20	DIS	1	paper	100.0	0.0	225	23.2
20	DIS	1	sliding	100.0	0.0	45	7.1
30	DIS	20%	paper	100.0	0.0	225	35.4
30	DIS	20%	sliding	100.0	0.0	45	15.3
40	DIS	30%	paper	100.0	0.0	225	51.4
40	DIS	30%	sliding	100.0	0.0	15	23.5
20	neighbour	1	paper	100.0	7.4	225	5.4
20	neighbour	1	sliding	100.0	52.6	29	5.4
30	neighbour	20%	paper	33.3	1.7	225	9.8
30	neighbour	20%	sliding	46.7	12.5	25	9.8
40	neighbour	30%	paper	5.0	0.0	225	20.6
40	neighbour	30%	sliding	13.3	1.4	11	20.5

Table 1: Main matrix, node-level, mean over 5 seeds. DIO/min is DIO receptions per monitoring node per minute over the full 30-minute run, counted from the per-message log so that both modes are measured over the same span. Attack-free baselines are given per size, since the rate grows with density

Both modes share identical counters, thresholds, topologies and seeds; they differ only in window length and evaluation cadence. Averaged over the six attack configurations the sliding window cuts mean detection latency from 225 s to 28 s, a factor of eight, while raising mean node-level FPR from 1.5% to 11.1% and mean node-level TPR from 73.1% to 76.7% (Figure 4).

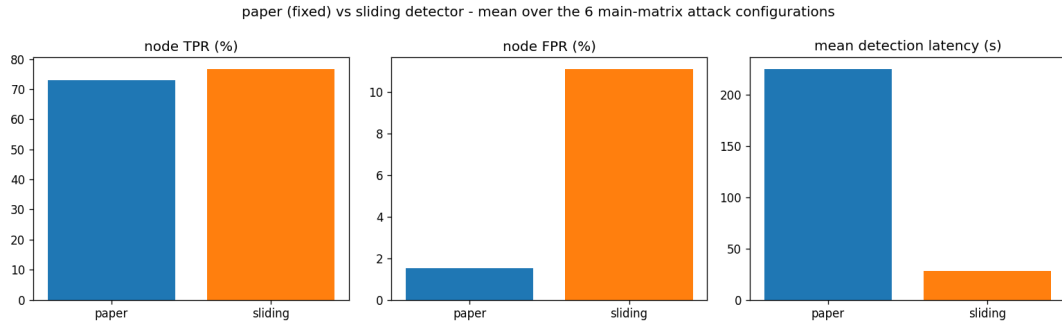


Figure 4: Fixed versus sliding window, averaged over the six main-matrix attack configurations: detection rate, false-positive rate and mean latency

Two results were not anticipated. The sliding window *improves* TPR under multiple attackers, from 33% to 47% and from 5% to 13%, because a shorter window captures an attacker’s burst before the other attackers’ counts accumulate into the mean; it therefore partly resists the masking of Finding 2. It also cuts the traffic the DIS attack provokes by more than half — from 51.4 to 23.5 DIO receptions per node per minute in the largest configuration, and to 31% and 43% of the paper-mode figure at 20 and 30 nodes — because attackers are blocked earlier and reset far fewer Trickle timers. Notably the effect is specific to the DIS attack: for the neighbour attack the two modes are indistinguishable (5.4 against 5.4, 9.8 against 9.8, 20.6 against 20.5), because a neighbour attacker’s cost is its own DIOs, which early blocking cannot retract. A faster detector is not only more responsive but cheaper for the network it protects, and only against the attack that recruits that network.

The costs are equally concrete, and both are cautionary. In attack-free runs node-level FPR reaches 99–100% against 41–51%, simply because evaluating thirty times more often gives each honest node thirty times more chances to be flagged once by quantisation. More seriously, the sliding mode issues **97 permanent blocks per run at 20 nodes, 141 at 30 and 188 at 40, with no attacker**

present, against effectively none in paper mode (one across fifteen baseline runs), because the response parameters (two temporary blocks, then permanent) were calibrated for one decision per five minutes and not for thirty. A sliding detector cannot inherit them unchanged.

5.4 Sensitivity to Threshold and Attack Rate

The DIS threshold was tested at 2, 3 and 5, at the paper’s random 5–60 s attack rate: TPR remained 100% and DIS-rule FPR 0% in all three, with only the latency rising at 5 (325 s against 225 s). Detection is insensitive here because honest nodes never exceed 2 per window and the attacker delivers 5–9, so any value between separates them. The paper’s choice of 3 sits comfortably in the usable range.

The attack-rate sweep (Table 2) produced a **third finding**. The paper mode detects every rate with the same one-window latency, which is both its strength and its weakness. The sliding mode scales with the attack, down to 5 s, but *fails completely against a slow DIS attacker*: one message every 30 s puts at most two inside a 60 s window, and two never exceeds a threshold of 3 that was calibrated on a window five times longer. Note that this is caused by the shorter *window*, not by the faster cadence: evaluating more often cannot recover a count the window is too short to accumulate. The lesson generalises beyond this project: **a rule expressed as an absolute count is bound to the window that produced it**, and shortening the window silently weakens it. The neighbour rule suffers nothing of the sort because it compares a neighbour against its peers rather than against a fixed number, and it holds at every rate in both modes.

Attack	Period	Paper mode	Sliding mode
DIS	1 s	100% / 225 s	100% / 5 s
DIS	5 s	100% / 225 s	100% / 25 s
DIS	10 s	100% / 225 s	100% / 45 s
DIS	30 s	100% / 225 s	0% / not detected
DIS	random 5–60 s	100% / 225 s	33% / 575 s
neighbour	1 s	100% / 225 s	100% / 5 s
neighbour	5 s	100% / 225 s	100% / 22 s
neighbour	10 s	100% / 225 s	100% / 28 s
neighbour	30 s	100% / 225 s	100% / 55 s
neighbour	random 5–60 s	100% / 225 s	100% / 38 s

Table 2: TPR / mean latency as a function of attack rate, 20 nodes, one attacker, mean over 3 seeds

5.5 Network Impact and Resource Cost

Packet delivery is effectively perfect in every configuration, including twelve attackers out of forty: 156 of the 168 runs deliver every datagram and no run loses more than a single one, the worst delivering 1124 of 1125 (PDR 0.9991). End-to-end delay is unchanged by the attacks and tracks only network size (38 ms at 20 nodes, 50 ms at 30, 63 ms at 40). This is a limitation of the setup rather than a property of the attacks: the UDGM radio is configured with success ratio 1.0 and one datagram per node every 30 s leaves the channel far from saturation. A lossy model would give a less favourable picture.

Two things follow, and the second corrects an expectation the first invites. The damage falls entirely on the control plane, and there it is severe: measured against the attack-free baseline *at the same network size*, a single DIS attacker multiplies DIO receptions by 5.6, 20% of attackers by 7.7 and 30% by 10.5 (Table 1). On battery-powered devices that translates directly into shortened lifetime. But the routing plane is *not* disturbed: parent changes per node are 1.000 in 153 of the 168 runs and never exceed 1.105, in attack-free and attacked runs alike. Every node selects a parent once and keeps it. The DIS attack multiplies control traffic tenfold without displacing a single routing decision — these attacks are an energy attack, not a routing attack, in this setting. The observation is worth stating plainly, in both directions: *an attack can be entirely successful on the metric that matters while the two metrics one instinctively checks, delivery and stability, stay perfect*.

On the Tmote Sky the detector module needs 1988 B of ROM and 588 B of RAM in paper mode, and

2204 B / 1070 B in sliding mode, the extra 482 B being the per-neighbour ring buffers; the paper’s claim of a lightweight module is confirmed. The platform, however, is the binding constraint. Under Contiki-NG v5.2 a stock node with only the routing layer and a UDP application already occupies 44578 of 49152 B of ROM (91%), leaving 4574 B. The detector fits that margin, but the instrumented firmware used for these experiments overflows ROM by 11 kB. Removing the project’s own CSV logging recovers only 868 B; what actually costs the space is the raised Contiki-NG log levels this project sets to make runs debuggable, which compile format strings and logging paths into the whole IPv6/RPL stack. With both removed the detector node does link, at 44482 of 49152 B of ROM (90.5%) and 7202 of 10240 B of RAM. A deployment-shaped build is therefore Sky-viable and was verified to be so; the instrumented one is not, which is why the experiments run on the native `cooja` mote. This is a finding about the platform, not the method: the paper used the same hardware on Contiki 2.7, whose routing layer was much smaller.

6 Conclusion

A distributed anomaly-based IDS can detect RPL neighbour and DIS attacks from purely local observations, and the reference model reproduces on a modern stack a decade after publication. The DIS attack is detected at 100% TPR with 0% DIS-rule FPR at every size and proportion tested, and a single neighbour attacker is detected reliably, at a cost of about 2 kB of ROM, 0.6 kB of RAM and no added traffic whatsoever, since the detector transmits nothing.

The reproduction also bounds those claims. Because the threshold computes its dispersion over a set containing the attacker, it can never detect a lone attacker at certain neighbourhood sizes, and it degrades sharply when attackers share a monitor: TPR falls from 100% to 33% to 5% as the attacker proportion rises. Both effects were predicted analytically and the first confirmed in simulation; the second predicted band lies outside the neighbourhood sizes this topology can produce, and rests on extrapolating the paper’s polynomial beyond the data it was fitted to, which is itself a finding about the published coefficient. Neither effect is visible in an evaluation reporting only aggregate rates.

The sliding-window extension shows a clear trade-off: latency falls by a factor of eight and, unexpectedly, masking resistance improves and the DIS-induced traffic more than halves; against this, false positives rise sharply, the response parameters over-block the network unless re-derived, and a slow DIS attacker escapes entirely. The last point is the most transferable result of the work: **the parameters of a threshold detector cannot be separated from the observation window they were measured on.**

Against the hypotheses set out at the start of the project:

- **H1** — *a node sending excessive DIOs deviates from the neighbourhood profile and is detected.* Holds only for a single attacker, and only at monitors outside the blind neighbourhood sizes. Qualified.
- **H2** — *a node sending excessive DIS is detected when its count exceeds the per-window threshold.* Confirmed for the paper’s window, and shown to be conditional on it: the same rule fails against a 30 s attacker once the window is shortened to 60 s.
- **H3** — *more attackers means more attack traffic, but also more false positives and more routing instability.* Only the first clause holds. False positives *fall* as the proportion rises (7.4%, 1.7%, 0.0% for the neighbour rule), because additional attackers raise the threshold for everyone — the same mechanism as Finding 2, seen from the other side — and routing instability does not appear at all (1.00 parent changes per node throughout). The hypothesis is refuted in both of its second clauses, and what replaces it is more interesting than what it predicted.
- **H4** — *a sliding window reduces latency at the price of false positives.* Confirmed, with the unanticipated addition that it also improves TPR under collusion and reduces DIS-induced traffic.
- **H5** — *measurable memory and code cost, no centralised monitoring.* Confirmed for memory and code, and the communication cost turns out to be exactly zero; decentralisation is what makes the detector work at all, since it is a non-blind monitor somewhere in the neighbourhood that rescues Finding 1.